



# UML Diagramming: A Case Study Approach



# Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

# UML Diagramming

A Case Study Approach

Dr. Suriya Sundaramoorthy



**CRC Press**

Taylor & Francis Group

AN AUERBACH BOOK

First edition published 2022  
by CRC Press  
6000 Broken Sound Parkway NW, Suite 300, Boca Raton, FL 33487-2742

and by CRC Press  
4 Park Square, Milton Park, Abingdon, Oxon, OX14 4RN

© 2022 Taylor & Francis Group, LLC

CRC Press is an imprint of Taylor & Francis Group, LLC

Reasonable efforts have been made to publish reliable data and information, but the author and publisher cannot assume responsibility for the validity of all materials or the consequences of their use. The authors and publishers have attempted to trace the copyright holders of all material reproduced in this publication and apologize to copyright holders if permission to publish in this form has not been obtained. If any copyright material has not been acknowledged please write and let us know so we may rectify in any future reprint.

Except as permitted under U.S. Copyright Law, no part of this book may be reprinted, reproduced, transmitted, or utilized in any form by any electronic, mechanical, or other means, now known or hereafter invented, including photocopying, microfilming, and recording, or in any information storage or retrieval system, without written permission from the publishers.

For permission to photocopy or use material electronically from this work, access [www.copyright.com](http://www.copyright.com) or contact the Copyright Clearance Center, Inc. (CCC), 222 Rosewood Drive, Danvers, MA 01923, 978-750-8400. For works that are not available on CCC please contact [mpkbookspermissions@tandf.co.uk](mailto:mpkbookspermissions@tandf.co.uk)

*Trademark notice:* Product or corporate names may be trademarks or registered trademarks and are used only for identification and explanation without intent to infringe.

ISBN: 978-1-032-26129-4 (hbk)  
ISBN: 978-1-032-12078-2 (pbk)  
ISBN: 978-1-003-28712-4 (ebk)

DOI: 10.1201/9781003287124

Typeset in Garamond  
by codeMantra

In the loving memory of my paternal grandfather Mr. S. Shanmugavelayutham (Telecom Employee), who taught me how much important is to educate a girl child in a family.

In the loving memory of my late maternal grandmother Mrs. A. Shenbagalakshmi (School Teacher) and late paternal grandmother Mrs. S. Sethulakshmi (Home Maker), who both taught me that the life of a working mother is actually a life of a warrior.

To my father, Mr. S. Sundaramoorthy (Postal Employee), who always have faith, love and hope on me with a strong belief that he has given enough education to his daughter to make a better tomorrow for herself.

To my mother, Mrs. R. J. Shanthi (BSNL Employee), who motivated me always to break all my hurdles.

To my loving daughters Shriya and Sana Dheepthi.

To my well-wishers and Gurus: Dr. V. C. S. Immanuel,  
Dr. S. P. Shantharajah and Mr. Ganesh Natrajan

To all my dear students who make me learn every time especially the batches of 2015–2019, 2016–2020, 2017–2021 and 2018–2022 of PSG College of Technology, Coimbatore, India who assisted me in this regard.

Finally, I would like to thank the great Almighty for blessing me with all these wonderful superheroes around me.



# Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

---

# Contents

---

|                                                                                                   |             |
|---------------------------------------------------------------------------------------------------|-------------|
| <b>Preface .....</b>                                                                              | <b>xi</b>   |
| <b>Author.....</b>                                                                                | <b>xiii</b> |
| <br>                                                                                              |             |
| <b>1 Introduction to UML Diagrams and Its Components .....</b>                                    | <b>1</b>    |
| 1.1 UP Phases.....                                                                                | 1           |
| 1.1.1 UML.....                                                                                    | 2           |
| 1.1.2 UML Usecase Diagram .....                                                                   | 4           |
| 1.1.2.1 Usecase Template .....                                                                    | 6           |
| 1.1.3 UML Activity Diagram .....                                                                  | 6           |
| 1.1.4 UML Sequence Diagram .....                                                                  | 10          |
| 1.1.5 UML Collaboration Diagram .....                                                             | 13          |
| 1.1.6 UML State Chart Diagram.....                                                                | 14          |
| 1.1.7 UML Class Diagram .....                                                                     | 17          |
| 1.1.8 UML Component Diagram.....                                                                  | 20          |
| 1.1.9 UML Deployment Diagram .....                                                                | 21          |
| <br>                                                                                              |             |
| <b>2 Design of UML Diagrams for Webmed – Healthcare Service System Services.....</b>              | <b>23</b>   |
| 2.1 Problem Statement .....                                                                       | 23          |
| 2.2 Major Functionalities and Its Description .....                                               | 24          |
| 2.3 Data-Flow Diagrams .....                                                                      | 25          |
| 2.4 Proposed Design – UML Diagrams .....                                                          | 27          |
| 2.4.1 Usecase Specification for the UML Usecase Diagram Generated<br>Using Star UML Tool.....     | 27          |
| <br>                                                                                              |             |
| <b>3 Design of UML Diagrams for Inventory Management System.....</b>                              | <b>39</b>   |
| 3.1 Problem Statement .....                                                                       | 39          |
| 3.2 Software Requirements Specification.....                                                      | 39          |
| 3.3 Functional Requirements.....                                                                  | 40          |
| 3.4 Design and Design Constraints.....                                                            | 40          |
| 3.5 UML Diagrams.....                                                                             | 46          |
| 3.6 Code Generated.....                                                                           | 54          |
| 3.6.1 Inventree.java .....                                                                        | 54          |
| 3.6.2 AddGoods.java.....                                                                          | 55          |
| 3.6.3 Screenshots of the Project.....                                                             | 56          |
| <br>                                                                                              |             |
| <b>4 Design of UML Diagrams for Business Process Outsourcing (BPO)<br/>Management System.....</b> | <b>59</b>   |
| 4.1 Problem Statement .....                                                                       | 59          |



|           |                                                                       |            |
|-----------|-----------------------------------------------------------------------|------------|
| 4.2       | Functional Requirements.....                                          | 59         |
| 4.3       | Design and Design Constraints.....                                    | 60         |
| <b>5</b>  | <b>Design of UML Diagrams for E-Ticketing for Buses .....</b>         | <b>75</b>  |
| 5.1       | Problem Statement .....                                               | 75         |
| 5.2       | UML Diagrams.....                                                     | 76         |
| <b>6</b>  | <b>Design of UML Diagrams for Weather Monitoring System .....</b>     | <b>85</b>  |
| 6.1       | Problem Statement .....                                               | 85         |
| 6.2       | Data Flow Diagrams .....                                              | 85         |
| 6.3       | UML Diagrams.....                                                     | 87         |
| <b>7</b>  | <b>Design of UML Diagrams for E-Province.....</b>                     | <b>95</b>  |
| 7.1       | Problem Statement .....                                               | 95         |
| 7.2       | UML Diagrams.....                                                     | 102        |
| <b>8</b>  | <b>Design of UML Diagrams for DigiDocLocker .....</b>                 | <b>105</b> |
| 8.1       | Problem Statement .....                                               | 105        |
| 8.2       | UML Diagrams.....                                                     | 106        |
| <b>9</b>  | <b>Design of UML Diagrams for Online Marketplace .....</b>            | <b>113</b> |
| 9.1       | Problem Statement .....                                               | 113        |
| 9.2       | UML Diagrams.....                                                     | 114        |
| <b>10</b> | <b>Design of UML Diagrams for Product Recommendation System .....</b> | <b>123</b> |
| 10.1      | Problem Statement .....                                               | 123        |
| 10.2      | UML Diagrams.....                                                     | 123        |
| <b>11</b> | <b>Design of UML Diagrams for Advocate Diary.....</b>                 | <b>129</b> |
| 11.1      | Problem Statement .....                                               | 129        |
| 11.2      | UML Diagrams.....                                                     | 129        |
| <b>12</b> | <b>Design of UML Diagrams for My Helper .....</b>                     | <b>139</b> |
| 12.1      | Problem Statement .....                                               | 139        |
| 12.2      | UML Diagrams.....                                                     | 141        |
| <b>13</b> | <b>Design of UML Diagrams for COVID-19 Management System.....</b>     | <b>147</b> |
| 13.1      | Problem Statement .....                                               | 147        |
| 13.2      | UML Diagrams.....                                                     | 149        |
| <b>14</b> | <b>Design of UML Diagrams for Car Care.....</b>                       | <b>157</b> |
| 14.1      | Problem Statement .....                                               | 157        |
| 14.2      | UML Diagrams.....                                                     | 158        |
| <b>15</b> | <b>Design of UML Diagrams for E-Ration Shop .....</b>                 | <b>167</b> |
| 15.1      | Problem Statement .....                                               | 167        |
| 15.2      | UML Diagrams.....                                                     | 168        |
| <b>16</b> | <b>Design of UML Diagrams for Textile Management System.....</b>      | <b>177</b> |
| 16.1      | Problem Statement .....                                               | 177        |
| 16.2      | UML Diagrams.....                                                     | 177        |

|           |                                                                           |            |
|-----------|---------------------------------------------------------------------------|------------|
| <b>17</b> | <b>Design of UML Diagrams for National Health ID 2020 .....</b>           | <b>185</b> |
| 17.1      | Problem Statement .....                                                   | 185        |
| 17.2      | UML Diagrams.....                                                         | 185        |
| <b>18</b> | <b>Design of UML Diagrams for Device Handout System .....</b>             | <b>191</b> |
| 18.1      | Problem Statement .....                                                   | 191        |
| 18.2      | UML Diagrams.....                                                         | 191        |
| <b>19</b> | <b>Design of UML Diagrams for Online College Magazine System.....</b>     | <b>199</b> |
| 19.1      | Problem Statement .....                                                   | 199        |
| 19.2      | UML Diagrams.....                                                         | 204        |
| <b>20</b> | <b>Design of UML Diagrams for Crime Bureau .....</b>                      | <b>209</b> |
| 20.1      | Problem Statement .....                                                   | 209        |
| 20.2      | UML Diagrams.....                                                         | 209        |
| <b>21</b> | <b>Design of UML Diagrams for Smart Traffic Management System .....</b>   | <b>219</b> |
| 21.1      | Problem Statement .....                                                   | 219        |
| 21.2      | UML Diagrams.....                                                         | 219        |
| <b>22</b> | <b>Design of UML Diagrams for Job Seeker Portal System.....</b>           | <b>227</b> |
| 22.1      | Problem Statement .....                                                   | 227        |
| 22.2      | UML Diagrams.....                                                         | 227        |
| <b>23</b> | <b>Design of UML Diagrams for AAROGYA SETU – Health Care APP .....</b>    | <b>237</b> |
| 23.1      | Problem Statement .....                                                   | 237        |
| 23.2      | UML Diagrams.....                                                         | 238        |
| <b>24</b> | <b>Design of UML Diagrams for Online Pharmacy Management System .....</b> | <b>247</b> |
| 24.1      | Problem Statement .....                                                   | 247        |
| 24.2      | UML Diagrams.....                                                         | 247        |
| <b>25</b> | <b>Design of UML Diagrams for EQUIHEALTH.....</b>                         | <b>265</b> |
| 25.1      | Problem Statement .....                                                   | 265        |
| 25.2      | System Architecture .....                                                 | 267        |
| 25.3      | UML Diagrams.....                                                         | 268        |
| <b>26</b> | <b>Design of UML Diagrams for an OTT-Based System – MINI REEL.....</b>    | <b>279</b> |
| 26.1      | Problem Statement .....                                                   | 279        |
| 26.2      | UML Diagrams.....                                                         | 280        |
| <b>27</b> | <b>Design of UML Diagrams for e-Med Medical Assistance Tool.....</b>      | <b>287</b> |
| 27.1      | Problem Statement .....                                                   | 287        |
| 27.2      | UML Diagrams.....                                                         | 287        |
| <b>28</b> | <b>Design of UML Diagrams for Diet Care.....</b>                          | <b>295</b> |
| 28.1      | Problem Statement .....                                                   | 295        |
| 28.2      | UML Diagrams.....                                                         | 295        |

|                    |                                                                                |            |
|--------------------|--------------------------------------------------------------------------------|------------|
| <b>29</b>          | <b>Design of UML Diagrams for Student Counselling Management System.....</b>   | <b>303</b> |
| 29.1               | Problem Statement .....                                                        | 303        |
| 29.2               | UML Diagrams.....                                                              | 303        |
| <b>30</b>          | <b>Design of UML Diagrams for E-Visa Processing and Follow-Up System .....</b> | <b>313</b> |
| 30.1               | Problem Statement .....                                                        | 313        |
| 30.2               | UML Diagrams.....                                                              | 319        |
| <b>31</b>          | <b>Design of UML Diagrams for Placement Automation System.....</b>             | <b>325</b> |
| 31.1               | Problem Statement .....                                                        | 325        |
| 31.2               | UML Diagrams.....                                                              | 326        |
| <b>32</b>          | <b>Design of UML Diagrams for Farm Management System .....</b>                 | <b>333</b> |
| 32.1               | Problem Statement .....                                                        | 333        |
| 32.2               | UML Diagrams.....                                                              | 333        |
| <b>33</b>          | <b>Design of UML Diagrams for Green Rides.....</b>                             | <b>341</b> |
| 33.1               | Problem Statement .....                                                        | 341        |
| 33.2               | UML Diagrams.....                                                              | 342        |
| <b>34</b>          | <b>Design of UML Diagrams for Art Gallery Management System .....</b>          | <b>349</b> |
| 34.1               | Problem Statement .....                                                        | 349        |
| 34.2               | UML Diagrams.....                                                              | 349        |
| <b>35</b>          | <b>Design of UML Diagrams for GUIDE – Dropshipping Website .....</b>           | <b>357</b> |
| 35.1               | Problem Statement .....                                                        | 357        |
| 35.2               | UML Diagrams.....                                                              | 357        |
| <b>36</b>          | <b>Design of UML Diagrams for Online Quiz System .....</b>                     | <b>367</b> |
| 36.1               | Problem Statement .....                                                        | 367        |
| 36.2               | UML Diagrams.....                                                              | 367        |
| <b>37</b>          | <b>Design of UML Diagrams for Book Bank Management System .....</b>            | <b>377</b> |
| 37.1               | Problem Statement .....                                                        | 377        |
| 37.2               | Literature Review .....                                                        | 378        |
| 37.3               | UML Diagrams.....                                                              | 378        |
| <b>38</b>          | <b>Design of UML Diagrams for Website Development.....</b>                     | <b>389</b> |
| 38.1               | Problem Statement .....                                                        | 389        |
| 38.2               | UML Diagrams.....                                                              | 389        |
| <b>39</b>          | <b>Design of UML Diagrams for STARTUP MEET .....</b>                           | <b>397</b> |
| 39.1               | Problem Statement .....                                                        | 397        |
| 39.2               | UML Diagrams.....                                                              | 397        |
| <b>40</b>          | <b>Design of UML Diagrams for Video Suggestion System.....</b>                 | <b>405</b> |
| 40.1               | Problem Statement .....                                                        | 405        |
| 40.2               | UML Diagrams.....                                                              | 405        |
| <b>Index .....</b> |                                                                                | <b>415</b> |

---

# Preface

---

*UML Diagramming: A Case Study Approach* is the outcome of efforts of bringing out the role of UML Diagrams in exposing the needs of any innovative idea. It covers 40 chapters focusing on development of UML diagrams to expose the requirements for better understanding of underlying systems. The various case studies taken for discussion are WEBMED – Healthcare Service System Services, Inventory Management System, Business Process Outsourcing (BPO) Management System, E-Ticketing for Buses, Weather Monitoring System, E-Province, DigiDocLocker, Online Marketplace, Product Recommendation System, Advocate Diary, My Helper, COVID-19 Management System, Car Care, E-Rationshop, Textile Management System, National Health ID 2020, Device Handout System Online College Magazine System, Crime Bureau, Smart Traffic Management System, Job Seeker Portal System, AAROGYA SETU – Health Care APP, Online Pharmacy Management System, EQUIHEALTH, an OTT-based System – MINI REEL, E-Med Medical Assistance Tool, Diet Care, Student Counselling Management System, Video Suggestion System, E-Visa Processing and Follow-Up System, Placement Automation System, Farm Management System, Green Rides, Art Gallery Management System, GUIDE – Dropshipping Website, Online Quiz System, Book Bank Management System, Website Development, START-UP MEET and Video Suggestion System.



# Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

---

# Author

---



**Dr. Suriya Sundaramoorthy** has completed her B.E. degree in the field of Computer Science and Engineering securing the 31st rank among all self-financing Engineering colleges under Anna University in 2004. She is a GATE scorer in GATE 2007. She is a gold medallist at ME CSE from PSG College of Technology, Coimbatore under Anna University in 2008. She was awarded with the BEST PROJECT AWARD for ME CSE stream. She was awarded with the BEST ALL ROUNDER of that batch. Later, she completed doctorate in the field of Information and Communication Engineering under Anna University in 2015. Currently, she is guiding Ph.D. scholars under Anna University. She has 10 years of experience in teaching. She has more than 50 journal and conference publications in the field of Grid Scheduling, Data Mining, Cloud Computing, Artificial

Intelligence and Computational Intelligence. She has nearly 10 book chapter publications in CRC Press, Springer and Elsevier.



# Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

# Chapter 1

---

## Introduction to UML Diagrams and Its Components

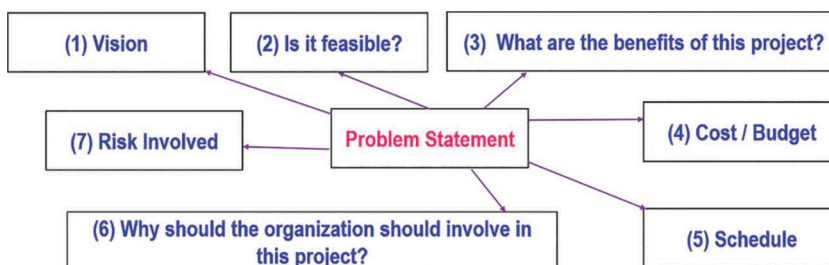
---

### 1.1 UP Phases

An approach that helps a user to build, deploy and maintain a software is termed as Software Development Process. Such a Software Development Process which is required to build an Object Oriented System is called as 'Unified Process'. Craig Larman states Unified Process as 'The Unified Process (UP) combines commonly accepted best practices, such as an iterative lifecycle and risk-driven development, into a cohesive and well-documented description/process'. There are four phases of the Unified Process, namely Inception, Elaboration, Construction and Transition. The inception phase takes a problem statement from a client as input. It traces answers for the below seven questions to generate a report to decide about the feasibility of developing the software (Figure 1.1).

The elaboration phase takes the report generated by the inception phase as input to perform its processing. It involves three main activities like developing a core architecture for the given system, developing a project schedule and finally sketching a Unified Modeling Language (UML) Usecase diagram (Figure 1.2).

The construction phase involves a set of three activities, namely, coding, testing and feedback. Notable point is that the construction phase involves Alpha testing, i.e. testing a software at the developer site (Figure 1.3).



---

Figure 1.1 Inception phase.



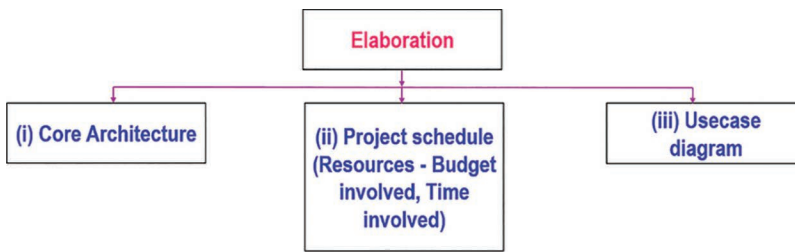


Figure 1.2 Elaboration phase.

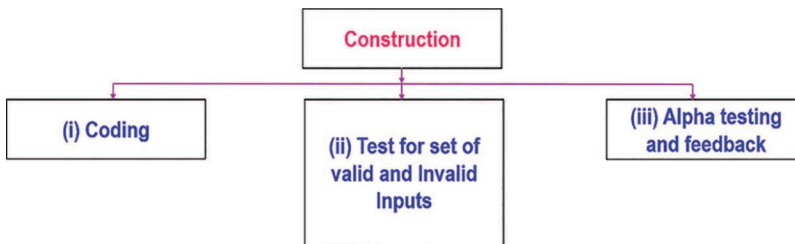


Figure 1.3 Construction phase.

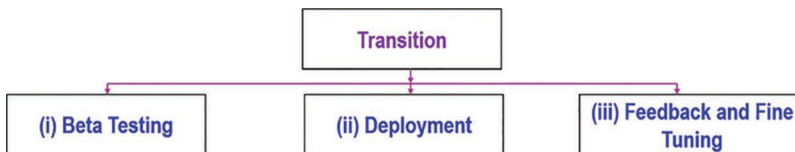


Figure 1.4 Transition phase.

The transition phase involves Beta testing, i.e. testing a software at the client site without the support of developers, followed by deployment, and finally, fine tuning is done based on feedback from the client's side (Figure 1.4).

### 1.1.1 UML

It is a standard diagrammatic tool for designing. It is a graphical language for specifying, visualizing, constructing and documenting the artefacts of software systems. It helps in better understanding of the software or system or product to be developed among developers and customers. UML diagrams include major important diagrams like UML Usecase Diagram, UML Activity Diagram, UML Class Diagram, UML Sequence Diagram, UML Collaboration Diagram or UML Communication Diagram, UML State Machine Diagram, UML Component Diagram and UML Deployment Diagram (Figure 1.5 and Table 1.1).

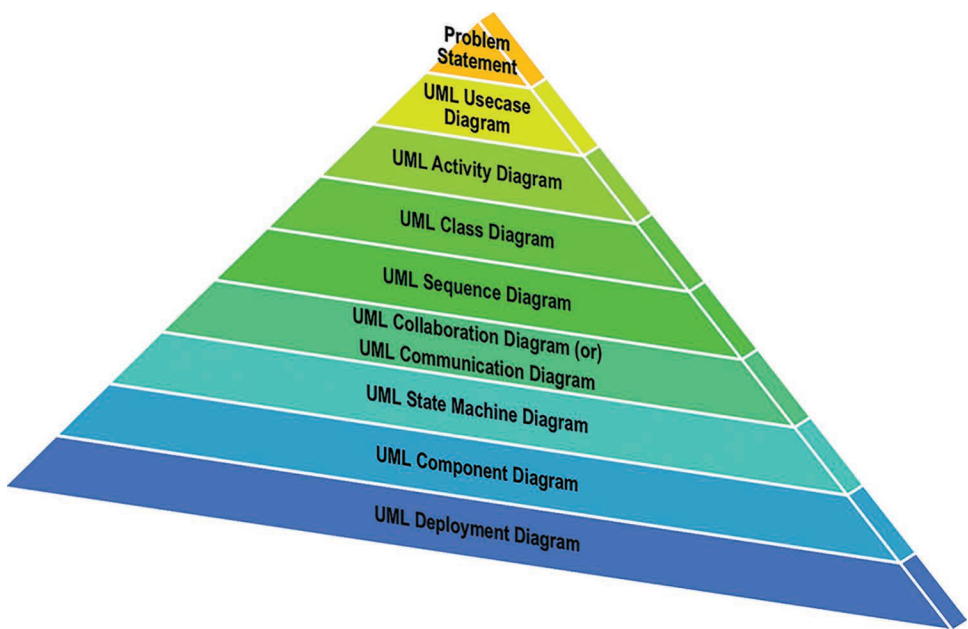


Figure 1.5 Pyramid of UML diagrams.

Table 1.1 List of UML Diagrams

| S. No | UML Diagram                                            | Purpose of the UML Diagram                                                                                                                             |
|-------|--------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1     | UML Usecase Diagram                                    | It focuses on the identification of functional requirements of the system under consideration.                                                         |
| 2     | UML Activity Diagram                                   | It focuses on sequential and parallel activities involved in each functional requirement of the system.                                                |
| 3     | UML Class Diagram                                      | It describes the structure of the system in terms of classes and objects.                                                                              |
| 4     | UML Sequence Diagram                                   | It depicts the objects involved in the scenario and the sequence of messages exchanged between the objects needed to carry out the functionality.      |
| 5     | UML Collaboration Diagram or UML Communication Diagram | It shows interactions between objects using sequenced messages in a free-form arrangement.                                                             |
| 6     | UML State Machine Diagram                              | It describes the life of an object using three main elements: states of an object, transitions between states and events that trigger the transitions. |

(Continued)

**Table 1.1 (Continued) List of UML Diagrams**

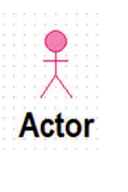
| S. No | UML Diagram            | Purpose of the UML Diagram                                                                                                                                                                                                                                                                                                                                                                              |
|-------|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7     | UML Component Diagram  | The purpose of a component diagram is to show the relationship between different components in a system.                                                                                                                                                                                                                                                                                                |
| 8     | UML Deployment Diagram | Deployment diagrams are used for describing the hardware components, where software components are deployed. The purpose of deployment diagrams can be described as: <ul style="list-style-type: none"> <li>• Visualize the hardware topology of a system.</li> <li>• Describe the hardware components used to deploy software components.</li> <li>• Describe the runtime processing nodes.</li> </ul> |

### 1.1.2 UML Usecase Diagram

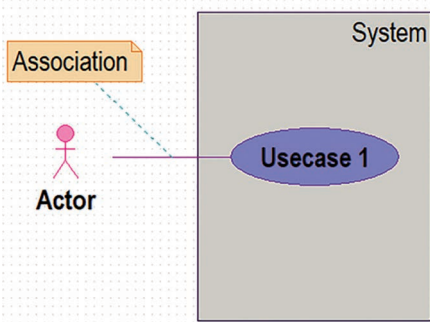
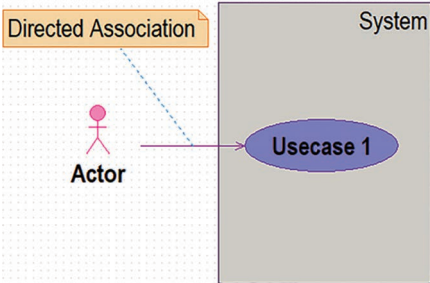
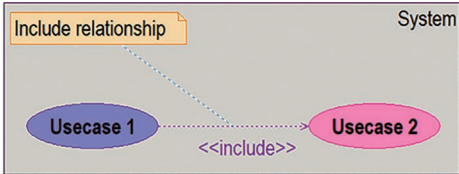
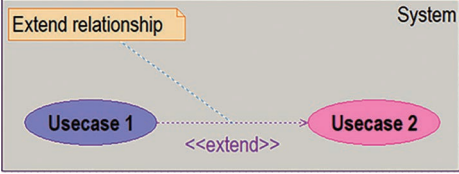
Table 1.2 contains the details of components of the UML Usecase Diagram.

Table 1.3 contains the various usecase relationships.

**Table 1.2 Components of UML Usecase Diagram**

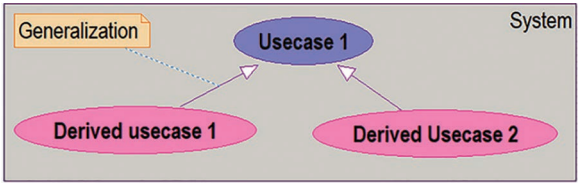
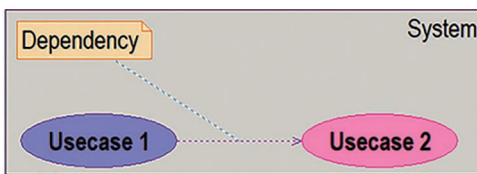
| S. No | Name of the Component | UML Notation                                                                        | Purpose                                                                                                                                                                                                                                                                                                                                                                                                           |
|-------|-----------------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1     | System Boundary       |  | <ul style="list-style-type: none"> <li>• It represents the scope of the system.</li> <li>• It encapsulates the complete set of functionalities of the system.</li> </ul>                                                                                                                                                                                                                                          |
| 2     | Actors                |  | <ul style="list-style-type: none"> <li>• The users that interact with a system.</li> <li>• An actor can be a person, an organization or an outside system that interacts with your application or system.</li> <li>• An actor in a usecase diagram interacts with a usecase.</li> <li>• It is up to the developer to consider what actors make an impact on the functionality that they want to model.</li> </ul> |
| 3     | Usecases              |  | <ul style="list-style-type: none"> <li>• A usecase is a visual representation of a distinct business functionality in a system.</li> <li>• It ensures that the business process is discrete in nature.</li> <li>• List the discrete business functions in given problem statement.</li> <li>• Identifying usecases is a discovery.</li> </ul>                                                                     |

**Table 1.3 Usecase Relationships**

| S. No | Usecase Relationship | UML Notation and Its Functionality                                                                                                                                                                                                                                                                                                     |
|-------|----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1     | Association          |  <ul style="list-style-type: none"> <li>• Association represents the relationship between an actor and a usecase.</li> </ul>                                                                                                                         |
| 2     | Directed Association |  <ul style="list-style-type: none"> <li>• Directed Association represents the one-way relationship where an actor takes the responsibility of influencing a usecase.</li> </ul>                                                                     |
| 3     | Include              |  <ul style="list-style-type: none"> <li>• Include relationship represents the situation where one usecase includes the functionality of one other usecase.</li> </ul>                                                                              |
| 4     | Extend               |  <ul style="list-style-type: none"> <li>• Extend relationship between any two usecases implies a meaningful relationship that the extended usecase adds up additional behaviour towards the existing functionality of the base usecase.</li> </ul> |

(Continued)

**Table 1.3 (Continued) Usecase Relationships**

| S. No | Usecase Relationship | UML Notation and Its Functionality                                                                                                                                                                                                                               |
|-------|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 5     | Generalization       |  <ul style="list-style-type: none"> <li>Generalization is the relationship between a parent usecase and one or more child usecases.</li> </ul>                                 |
| 6     | Dependency           |  <ul style="list-style-type: none"> <li>Dependency defines the relationship in which the existence of one usecase is dependent on the existence of another usecase.</li> </ul> |

### 1.1.2.1 Usecase Template

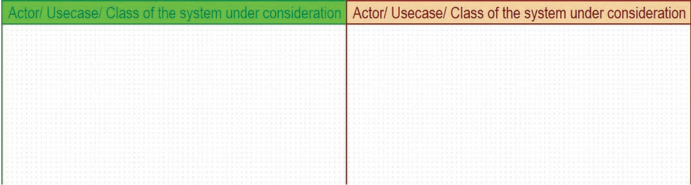
Every usecase is explained in detail through a standard template called as usecase template.

|                                                                                                 |
|-------------------------------------------------------------------------------------------------|
| <i>Usecase ID</i> – represents a unique ID for a usecase.                                       |
| <i>Usecase Name</i> – represents a meaningful name of the usecase.                              |
| <i>Actors Involved</i> – actors interacting with the usecase.                                   |
| <i>Description</i> – the brief explanation of the functionality of the usecase.                 |
| <i>Preconditions</i> – state of the system before executing this functionality of the usecase.  |
| <i>Main Flow</i> – description of how the functionality is implemented.                         |
| <i>Post Conditions</i> – state of the system after executing this functionality of the usecase. |
| <i>Alternate Flow</i> – other possibilities available.                                          |

### 1.1.3 UML Activity Diagram

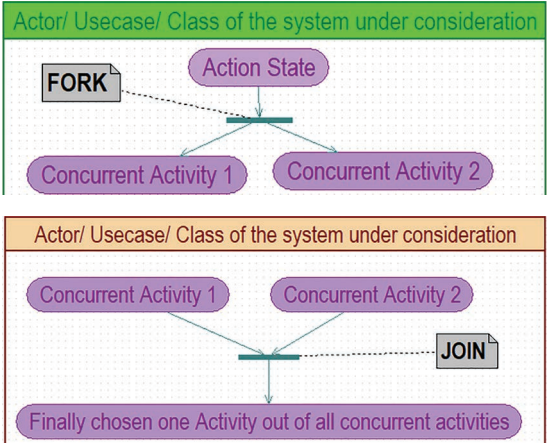
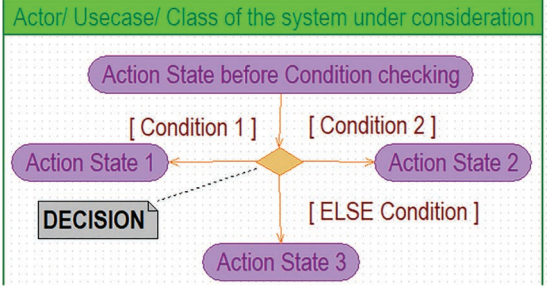
Table 1.4 contains the details of components of the UML Activity Diagram.

**Table 1.4 Components of UML Activity Diagram**

| S. No | Name of the Component | UML Notation and the Purpose of the Component                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|-------|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1     | Initial state         |  <ul style="list-style-type: none"> <li>• It represents the start state of the system under consideration.</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| 2     | Final state           |  <ul style="list-style-type: none"> <li>• It represents the termination state of the system under consideration.</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| 3     | Swimlanes             | <ul style="list-style-type: none"> <li>• A swimlane contains two partitions namely a top partition representing entities like an actor/a usecase/a class, etc. and the second partition focuses on the set of activities involved.</li> <li>• Swimlane is of two types namely vertical swimlane and horizontal swimlane.</li> <li>• Vertical swimlane – represents parallel activities of a particular scenario of the system under consideration.</li> </ul>  <ul style="list-style-type: none"> <li>• Horizontal swimlane – represents sequential activities of a particular scenario of the system under consideration.</li> </ul> |
| 4     | Action state          | <ul style="list-style-type: none"> <li>• An action state represents an operation or a business activity or a process.</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |

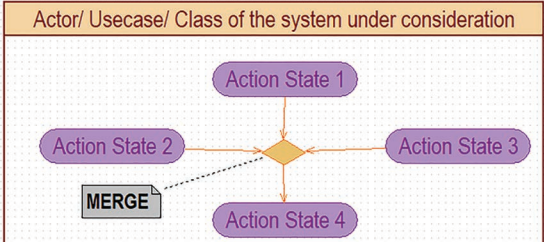
(Continued)

**Table 1.4 (Continued) Components of UML Activity Diagram**

| S. No | Name of the Component | UML Notation and the Purpose of the Component                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|-------|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 5     | Object                | <ul style="list-style-type: none"> <li>An entity that carries data between any two action states.</li> </ul>                                                                                                                                                                                                                                                                                                           |
| 6     | Synchronization       | <ul style="list-style-type: none"> <li>Synchronization represents two or more activities happening at same time or rate.</li> <li>It is of two types namely: Fork – divides a single activity flow into two or more concurrent activities and Join – combines two or more concurrent activities into a single flow by ensuring that only one of the activities at a time.</li> </ul>                                 |
| 7     | Decision              | <ul style="list-style-type: none"> <li>A decision node has one input or two or more outputs depending on the condition for which it is designed.</li> <li>Each output flow has a condition attached to it.</li> <li>If a condition is met, the flow proceeds along with the appropriate output.</li> <li>An 'else' output can be defined along which the flow can proceed if no other condition is met.</li> </ul>  |

(Continued)

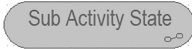
**Table 1.4 (Continued) Components of UML Activity Diagram**

| S. No | Name of the Component | UML Notation and the Purpose of the Component                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|-------|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 8     | Merge                 | <ul style="list-style-type: none"> <li>The purpose is to merge the input flows.</li> <li>The inputs are not synchronized; if a flow reaches such a node it proceeds at the output without waiting for the arrival of other flows.</li> </ul>                                                                                                                                                                                                                                                                                                  |
| 9     | Flow Final            | <ul style="list-style-type: none"> <li>It represents abnormal termination of a path in an activity diagram that is not considered as a part of the system under development.</li> </ul>                                                                                                                                                                                                                                                                                                                                                        |
| 10    | Transition            | <ul style="list-style-type: none"> <li>Transition are arrows that represent a movement from the source activity state to the target activity state which gets triggered by the completion of the activity of the source activity state.</li> </ul>                                                                                                                                                                                                                                                                                          |
| 11    | Self-Transition       | <ul style="list-style-type: none"> <li>It represents the internal transition to an actions state itself.</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                          |
| 12    | Signal Send State     | <ul style="list-style-type: none"> <li>Signals represent how activities can be influenced externally by the system.</li> <li>They usually appear in pairs of sent and received signals.</li> <li>Signal Send State represents sending signal action outside of the activity.</li> <li>The signal sends state does not wait for any responses from the receiver of the signal.</li> <li>It ends itself and passes the execution control to the next action.</li> <li>A Signal Send State takes its notation as a convex pentagon.</li> </ul>  |

(Continued)



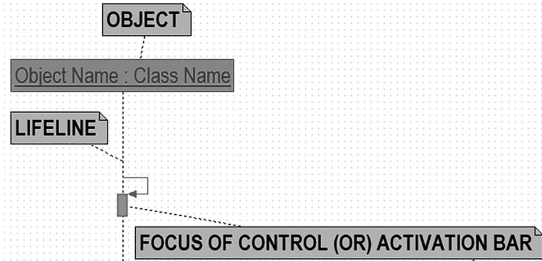
**Table 1.4 (Continued) Components of UML Activity Diagram**

| S. No | Name of the Component | UML Notation and the Purpose of the Component                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|-------|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 13    | Signal Accept State   | <ul style="list-style-type: none"> <li>• An action state whose trigger is a signal event is informally called Signal Accept State.</li> <li>• It corresponds to Signal Send State.</li> <li>• Signal Accept State with incoming edges means that the action starts after the previous action state completes.</li> <li>• Signal Accept State with no incoming edges remains enabled after it accepts an event.</li> <li>• It does not terminate after accepting an event and outputting a value, but continues to wait for other events.</li> </ul>  |
| 14    | Subactivity           | <ul style="list-style-type: none"> <li>• A subactivity is an action state that can become as another main activity scenario in future.</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                    |

### 1.1.4 UML Sequence Diagram

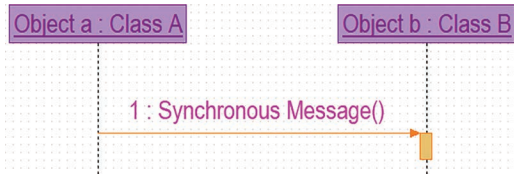
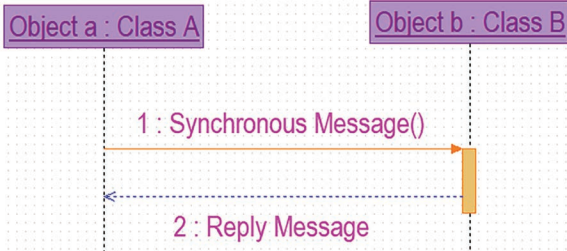
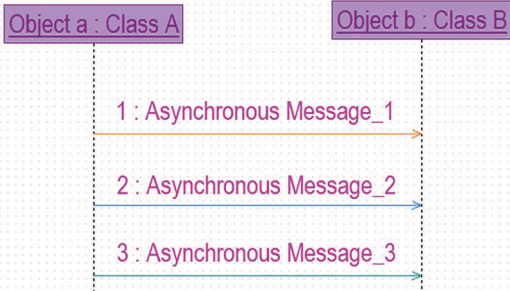
Table 1.5 contains the details of components of the UML Sequence Diagram.

**Table 1.5 Components of UML Sequence Diagram**

| S. No | Name of the Component | UML Notation and the Purpose of the Component                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|-------|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1     | Object                | <ul style="list-style-type: none"> <li>• It represents an entity of the system that gets involved in interaction with other entities of the system via messages.</li> <li>• An object has three related information namely object name, lifeline and focus of control.</li> <li>• The standard syntax of naming an object is Object_name: Class_name.</li> <li>• Lifeline represents the complete existence of a particular object involved in a scenario.</li> <li>• Focus of control represents the active session of the object.</li> </ul>  |

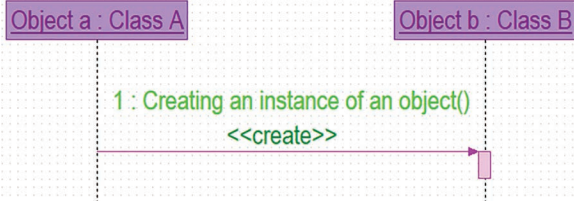
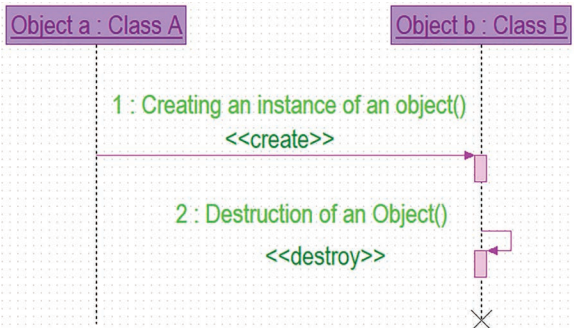
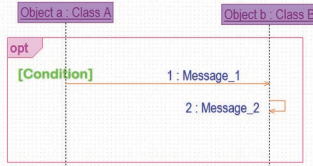
(Continued)

**Table 1.5 (Continued) Components of UML Sequence Diagram**

| S. No | Name of the Component | UML Notation and the Purpose of the Component                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|-------|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2     | Message & its types   | <ul style="list-style-type: none"> <li>The communication between objects of any scenario is done through the concept of message passing with the help of transition component.</li> <li>Messages are numbered over the transition arrows.</li> <li>Syntax of message is Message_no: Message()</li> <li>There are five types of messages applicable to any transition between any two objects in a sequence diagram.</li> </ul>                                                                                                                                                                                                                   |
| 2.1   | Synchronous Message   | <ul style="list-style-type: none"> <li>The messages sent from the sender are based on wait semantics.</li> <li>Wait semantics is a situation where the sender needs an acknowledgement of each message sent from the receiver before transmitting the next successive message.</li> <li>The notation of a synchronous message is a solid line with a shaded arrow head.</li> </ul>  <pre> sequenceDiagram     participant A as Object a : Class A     participant B as Object b : Class B     A-&gt;&gt;B: 1 : Synchronous Message()     activate B     </pre> |
| 2.2   | Return Message        | <ul style="list-style-type: none"> <li>It represents a reply message for a synchronous message.</li> </ul>  <pre> sequenceDiagram     participant A as Object a : Class A     participant B as Object b : Class B     A-&gt;&gt;B: 1 : Synchronous Message()     activate B     B--&gt;&gt;A: 2 : Reply Message     deactivate B     </pre>                                                                                                                                                                                                                   |
| 2.3   | Asynchronous Message  | <ul style="list-style-type: none"> <li>These type of messages are sent by the sender to the receiver which does not require an acknowledgement from the receiver.</li> <li>This feature enables the sender to send any number of messages to the receiver.</li> </ul>  <pre> sequenceDiagram     participant A as Object a : Class A     participant B as Object b : Class B     A-&gt;&gt;B: 1 : Asynchronous Message_1     A-&gt;&gt;B: 2 : Asynchronous Message_2     A-&gt;&gt;B: 3 : Asynchronous Message_3     </pre>                                  |

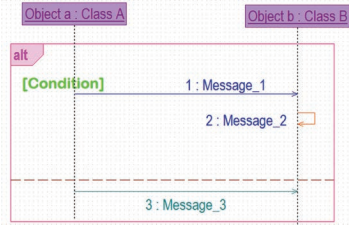
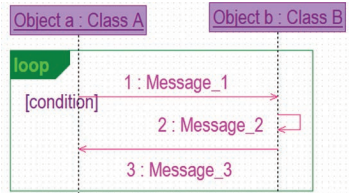
(Continued)

**Table 1.5 (Continued) Components of UML Sequence Diagram**

| S. No | Name of the Component | UML Notation and the Purpose of the Component                                                                                                                                                                                                                                                                                                                                                                                         |
|-------|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2.4   | Create Message        | <ul style="list-style-type: none"> <li>Create message represents the instantiation of objects in a scenario.</li> <li>It includes the stereotype called &lt;&lt;create&gt;&gt;.</li> </ul>                                                                                                                                                          |
| 2.5   | Destroy Message       | <ul style="list-style-type: none"> <li>Destroy message represents the destruction of the lifecycle of an object in a scenario.</li> <li>It includes the stereotype called &lt;&lt;destroy&gt;&gt;.</li> <li>Destruction is pictorially represented by a large X pointing at the end of the lifeline of an object.</li> </ul>                       |
| 3     | Fragments             | <ul style="list-style-type: none"> <li>Fragments represent a set of conditional messages or looping messages in a scenario.</li> <li>The most vital fragments are opt, alt, loop.</li> <li>Each fragment is represented as a rectangular box encapsulating those set of messages with a label in the top left corner representing the fragment type.</li> </ul>                                                                       |
| 3.1   | Opt                   | <ul style="list-style-type: none"> <li>This fragment represents a simple IF scenario.</li> <li>Messages get executed only if the condition defined in the opt fragment remains true.</li> <li>In case of condition does not get satisfied, the control jumps out of the fragment.</li> <li>Conditions are technically called as Guard.</li> </ul>  |

(Continued)

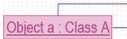
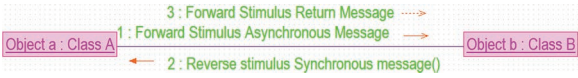
**Table 1.5 (Continued) Components of UML Sequence Diagram**

| S. No | Name of the Component | UML Notation and the Purpose of the Component                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|-------|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 3.2   | Alt                   | <ul style="list-style-type: none"> <li>This fragment represents IF ELSE scenario.</li> <li>This fragment has two partitions inside the rectangular area.</li> <li>Messages in the first compartment get executed only if the condition defined in the first partition of the alt fragment remains true.</li> <li>Otherwise, the messages in the second partition of the alt fragment get executed.</li> </ul>          |
| 3.3   | Loop                  | <ul style="list-style-type: none"> <li>This fragment represents those set of messages that gets executed repeatedly until the condition defined in the loop fragment remains true.</li> <li>This fragment has two partitions inside the rectangular area.</li> <li>Messages in the first compartment get executed only if the condition defined in the first partition of the alt fragment remains true.</li> </ul>  |
| 4     | Nesting of fragments  | <ul style="list-style-type: none"> <li>Depending on the demand of multiple conditions to get verified or repeated execution of the set of nested conditions, this concept of nesting can be applied to different types of fragments.</li> </ul>                                                                                                                                                                                                                                                         |

### 1.1.5 UML Collaboration Diagram

Majority of the components are common between UML Sequence Diagram and UML Collaboration Diagram. Table 1.6 contains the details of the unique components of the UML Collaboration Diagram.

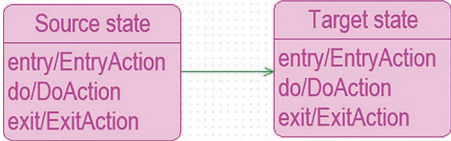
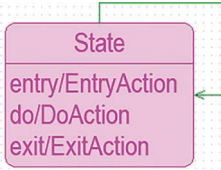
**Table 1.6 Unique Components of UML Collaboration Diagram**

| S. No | Name of the Component | UML Notation and the Purpose of the Component                                                                                                                                                                                                                                                                                                                                                                                         |
|-------|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1     | Object                | <ul style="list-style-type: none"> <li>It represents an entity of the system that gets involved in interaction with other entities of the system via messages.</li> <li>In the case of UML Collaboration diagrams, an object does not have any lifeline or focus of control.</li> </ul>                                                              |
| 2     | Link                  | <ul style="list-style-type: none"> <li>Link enables the communication between any two objects.</li> <li>It is represented as a solid line without any arrow heads, so that it represents bidirectional communication.</li> <li>Any number of bidirectional messages between two objects can be communicated via a same link.</li> </ul>              |
| 3     | Self Link             | <ul style="list-style-type: none"> <li>A self-link connects an object to itself.</li> </ul>                                                                                                                                                                                                                                                          |
| 4     | Forward stimulus      | <ul style="list-style-type: none"> <li>It represents messages from sender (first object) to receiver (second object).</li> <li>It can be any of the five types of messages as discussed in UML Sequence Diagram like Synchronous messages, Return messages, Asynchronous messages, Create messages and Destroy messages.</li> </ul>                |
| 5     | Reverse stimulus      | <ul style="list-style-type: none"> <li>It represents response messages from the receiver (second object) to sender (first object).</li> <li>It can be any of the five types of messages as discussed in UML Sequence Diagram like Synchronous messages, Return messages, Asynchronous messages, Create messages and Destroy messages.</li> </ul>  |

### 1.1.6 UML State Chart Diagram

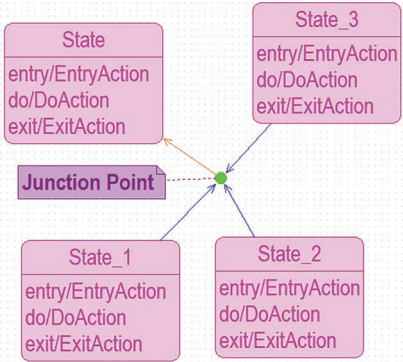
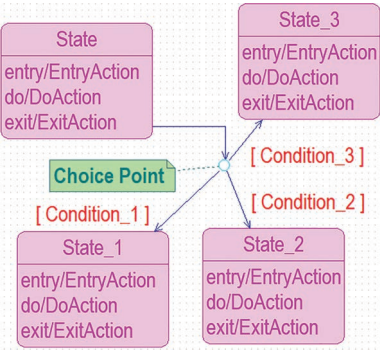
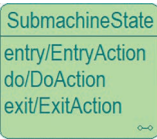
Table 1.7 contains the details of components of the UML State Machine Diagram.

**Table 1.7 Components of UML State Machine Diagram**

| S. No | Name of the Component | UML Notation and the Purpose of the Component                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|-------|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1     | Initial State         | <ul style="list-style-type: none"> <li>It represents the start state of a system under consideration.</li> </ul>                                                                                                                                                                                                                                                                                                                                                 |
| 2     | Final State           | <ul style="list-style-type: none"> <li>It represents the end state of a system under consideration.</li> </ul>                                                                                                                                                                                                                                                                                                                                                   |
| 3     | Flow final            | <ul style="list-style-type: none"> <li>It represents abnormal termination of a path in a state machine diagram which is not considered as a part of the system under development.</li> </ul>                                                                                                                                                                                                                                                                     |
| 4     | State                 | <ul style="list-style-type: none"> <li>It represents the state of the object in the system under consideration.</li> <li>It is described using the following attributes namely:               <ol style="list-style-type: none"> <li>Name – Name of the object.</li> <li>Entry – Action to be done before to enter the system.</li> <li>Do – Action to be done in that state.</li> <li>Exit – Action to be required to exit the system.</li> </ol> </li> </ul>  |
| 5     | Transition            | <ul style="list-style-type: none"> <li>Transition are arrows that represent a movement from one state to the other state which gets triggered by the completion of the activity of the source state.</li> </ul>                                                                                                                                                                                                                                               |
| 6     | Self-Transition       | <ul style="list-style-type: none"> <li>It represents internal transition to a state itself.</li> </ul>                                                                                                                                                                                                                                                                                                                                                         |
| 7     | Junction Point        | <ul style="list-style-type: none"> <li>The purpose is to merge the input flows from different states</li> <li>The inputs are not synchronized; if a flow reaches such a state it proceeds at the output without waiting for the arrival of other flows from other states.</li> </ul>                                                                                                                                                                                                                                                              |

(Continued)

**Table 1.7 (Continued) Components of UML State Machine Diagram**

| S. No | Name of the Component | UML Notation and the Purpose of the Component                                                                                                                                                                                                                                                                                                                                           |
|-------|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|       |                       |                                                                                                                                                                                                                                                                                                        |
| 8     | Choice Point          | <ul style="list-style-type: none"> <li>• A Choice point has one input or two or more outputs depending on the condition for which it is designed.</li> <li>• Each output flow has a condition attached to it. If a condition is met, the flow proceeds along with the appropriate output.</li> </ul>  |
| 9     | Shallow History       | <ul style="list-style-type: none"> <li>• Shallow history of a state is a reference to the most recently visited state on the same hierarchy level.</li> </ul>                                                                                                                                        |
| 10    | Deep History          | <ul style="list-style-type: none"> <li>• Deep history of a state is a reference to the most recently visited simple state.</li> </ul>                                                                                                                                                                |
| 11    | Submachine state      | <ul style="list-style-type: none"> <li>• A submachine is a state that can become as another main state chart diagram scenario in future.</li> </ul>                                                                                                                                                  |



### 1.1.7 UML Class Diagram

Table 1.8 contains the details of components of the UML Class Diagram.

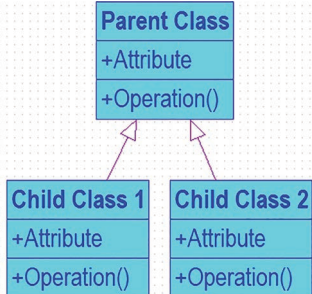
**Table 1.8 Components of UML Class Diagram**

| S. No               | Name of the Component | UML Notation and the Purpose of the Component                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |            |                  |              |                     |                  |              |              |   |           |   |         |
|---------------------|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|------------------|--------------|---------------------|------------------|--------------|--------------|---|-----------|---|---------|
| 1                   | Class                 | <div><ul style="list-style-type: none"><li>Each class has three compartments namely:<ul style="list-style-type: none"><li>i. Top compartment represents the name of class,</li><li>ii. Middle compartment represents structure of the class (attributes) and</li><li>iii. Bottom compartment represents behaviour of the class (operations)</li></ul></li></ul></div> <div><table><tr><td>Class_Name</td></tr><tr><td>+Attribute</td></tr><tr><td>+Operation()</td></tr></table></div> <div><ul style="list-style-type: none"><li>Attribute represents properties that describe the state of the object.</li><li>Syntax: Visibility Attribute_Name : Data_Type</li><li>Examples of data types are: Int, Char, Float, String, Date, Time, Money, Dollars, Colour, City, Country, Postal Code, Address, Boolean, etc.</li></ul></div> <div><table><tr><th>Visibility Notation</th><th>Access Specifier</th></tr><tr><td>+</td><td>Public</td></tr><tr><td>#</td><td>Protected</td></tr><tr><td>-</td><td>Private</td></tr></table></div> <div><ul style="list-style-type: none"><li>Derived attributes are those attributes that are calculated or derived from other attributes denoted by placing slash (/) before name.</li><li>Syntax:/Visibility Attribute_Name: Data_Type</li><li>Operations represents the actions or functions that a class can perform.</li><li>Syntax: Visibility Operation_Name (Input Arg): Data_Type</li><li>Input arguments take the syntax of an attribute.</li><li>Methods that don't return a value (i.e. void methods) should give a return type of void.</li></ul></div> | Class_Name | +Attribute       | +Operation() | Visibility Notation | Access Specifier | +            | Public       | # | Protected | - | Private |
| Class_Name          |                       |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |            |                  |              |                     |                  |              |              |   |           |   |         |
| +Attribute          |                       |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |            |                  |              |                     |                  |              |              |   |           |   |         |
| +Operation()        |                       |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |            |                  |              |                     |                  |              |              |   |           |   |         |
| Visibility Notation | Access Specifier      |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |            |                  |              |                     |                  |              |              |   |           |   |         |
| +                   | Public                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |            |                  |              |                     |                  |              |              |   |           |   |         |
| #                   | Protected             |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |            |                  |              |                     |                  |              |              |   |           |   |         |
| -                   | Private               |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |            |                  |              |                     |                  |              |              |   |           |   |         |
| 2                   | Class Relationships   | <ul style="list-style-type: none"><li>There are seven types of class relationship namely association, directed association, dependency, aggregation, composition, generalization, realization.</li></ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |            |                  |              |                     |                  |              |              |   |           |   |         |
| 2.1                 | Association           | <div><ul style="list-style-type: none"><li>When two classes communicate with each other, an association is used to connect those two classes.</li></ul></div> <div><table><tr><td>Class1</td><td rowspan="3">Association Name</td><td>Class 2</td></tr><tr><td>+Attribute</td><td>+Attribute</td></tr><tr><td>+Operation()</td><td>+Operation()</td></tr></table></div>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | Class1     | Association Name | Class 2      | +Attribute          | +Attribute       | +Operation() | +Operation() |   |           |   |         |
| Class1              | Association Name      | Class 2                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |            |                  |              |                     |                  |              |              |   |           |   |         |
| +Attribute          |                       | +Attribute                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |            |                  |              |                     |                  |              |              |   |           |   |         |
| +Operation()        |                       | +Operation()                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |            |                  |              |                     |                  |              |              |   |           |   |         |

(Continued)

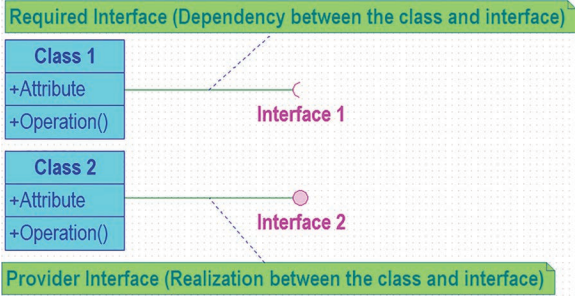
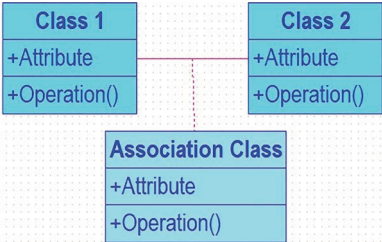


**Table 1.8 (Continued) Components of UML Class Diagram**

| S. No | Name of the Component | UML Notation and the Purpose of the Component                                                                                                                                                                                                                                                                                                                 |
|-------|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2.2   | Directed Association  | <ul style="list-style-type: none"> <li>When two classes get connected in such way by giving priority to one-way communication, they are connected by directed association.</li> </ul>                                                                                       |
| 2.3   | Dependency            | <ul style="list-style-type: none"> <li>When the existence of one class is dependent on the existence of another class, then they are connected using dependency.</li> </ul>                                                                                                 |
| 2.4   | Aggregation           | <ul style="list-style-type: none"> <li>It represents a 'part of' relationship.</li> <li>Many instances of Class1 can be associated with Class2.</li> <li>Aggregation implies a relationship where the child class can exist independently of the parent class.</li> </ul>  |
| 2.5   | Composition           | <ul style="list-style-type: none"> <li>It represents a 'whole' relationship.</li> <li>Composition implies a relationship where the child class cannot exist independent of the parent class.</li> </ul>                                                                   |
| 2.6   | Generalization        | <ul style="list-style-type: none"> <li>Generalization represents the concept of inheritance.</li> <li>It represents the relationship between parent class and its child classes.</li> </ul>                                                                                |

(Continued)

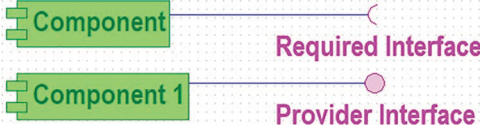
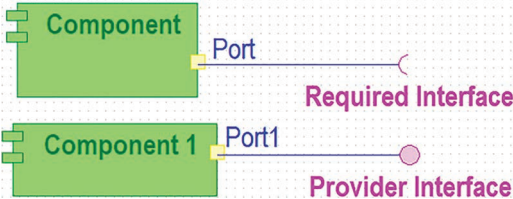
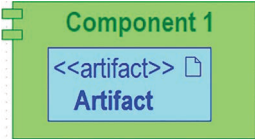
**Table 1.8 (Continued) Components of UML Class Diagram**

| S. No | Name of the Component           | UML Notation and the Purpose of the Component                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|-------|---------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2.7   | Interface & role of Realization | <ul style="list-style-type: none"> <li>• Interfaces can be of two types namely required interfaces and provider interfaces.</li> <li>• Provider interface describes the functionality offered by a class.</li> <li>• Required interfaces describe the functionality needed by another class.</li> <li>• Provided interface is the interface implemented by a class, i.e. a class implements an interface.</li> <li>• The required interface would be any use of an interface by a component, i.e. if a class defines a method that has the interface as a parameter.</li> </ul>  <p><b>Required Interface (Dependency between the class and interface)</b></p> <p>Class 1 (with +Attribute and +Operation()) is connected to Interface 1 (with a semi-circular dependency arrow).</p> <p>Class 2 (with +Attribute and +Operation()) is connected to Interface 2 (with a solid line and a circle at the interface end).</p> <p><b>Provider Interface (Realization between the class and interface)</b></p> |
| 3     | Association Class               | <ul style="list-style-type: none"> <li>• An association class is an association that is also a class.</li> <li>• It not only connects a set of classifiers but also defines a set of features that belong to the relationship itself and not any of the classifiers.</li> </ul>  <p>Class 1 (with +Attribute and +Operation()) and Class 2 (with +Attribute and +Operation()) are connected by a solid association line. An Association Class (with +Attribute and +Operation()) is connected to this association line with a dashed line.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                           |

### 1.1.8 UML Component Diagram

Table 1.9 contains the details of components of the UML Component Diagram.

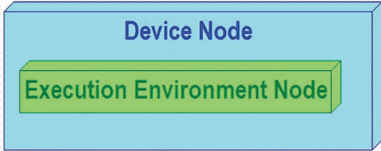
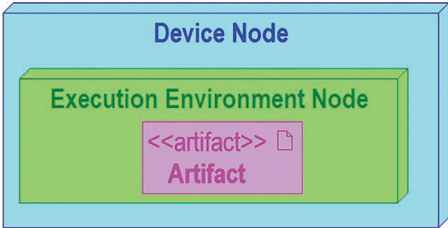
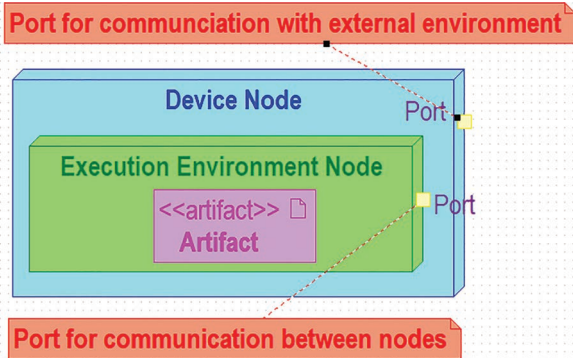
**Table 1.9 Components of UML Component Diagram**

| S. No | Name of the Component | UML Notation and the Purpose of the Component                                                                                                                                                                                                                                                                                                                                                           |
|-------|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1     | Component             | <ul style="list-style-type: none"> <li>It is a modular part of a system that encapsulates its contents.</li> <li>They are the logical elements of a system that plays an essential role during the execution of a system.</li> <li>A component is a replaceable and executable piece of a system.</li> </ul>           |
| 2     | Interface             | <ul style="list-style-type: none"> <li>There are two types of component interfaces, namely provider interface and required interface.</li> <li>Provider interface describes the functionality offered by a component.</li> <li>Required interfaces describe the functionality needed by another component.</li> </ul>  |
| 3     | Port                  | <ul style="list-style-type: none"> <li>A port enables better communication between an interface and a component.</li> </ul>                                                                                                                                                                                          |
| 4     | Artifact              | <ul style="list-style-type: none"> <li>Scripting files like PHP, database files, configuration files, rar files, executable files.</li> </ul>                                                                                                                                                                        |
| 5     | Association           | <ul style="list-style-type: none"> <li>It represents the two-way communication between any two components involved in a relationship.</li> </ul>                                                                                                                                                                     |

### 1.1.9 UML Deployment Diagram

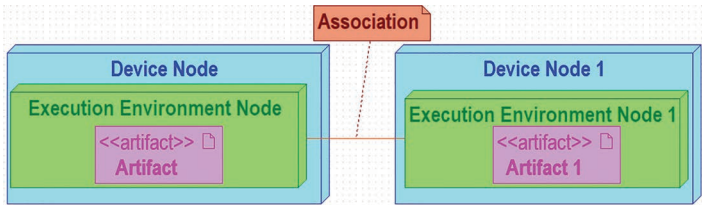
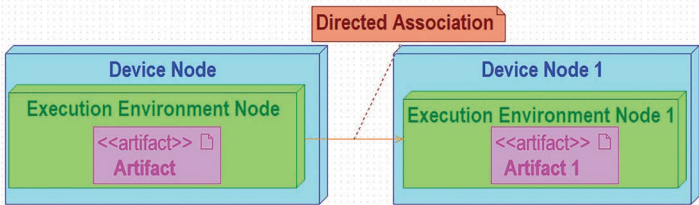
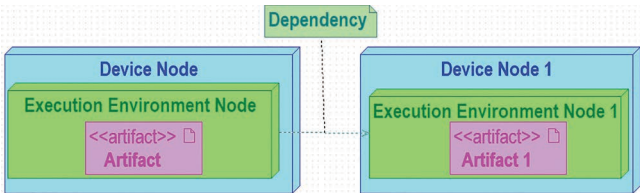
Table 1.10 contains the details of components of the UML Deployment Diagram.

**Table 1.10** Components of UML Deployment Diagram

| S. No | Name of the Component | UML Notation and the Purpose of the Component                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|-------|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1     | Node                  | <ul style="list-style-type: none"> <li>Any computational resource with memory and processing capability.</li> <li>There are two types of nodes namely Device node and Execution Environment Node.</li> <li>A Device node represents a hardware device like Application server, workstations, mobile device, embedded device, etc.</li> <li>An Execution Environment node represents a software or a program like Operating system, workflow engine, browser, Container, web server, database system, etc.</li> </ul>  |
| 2     | Artifact              | <ul style="list-style-type: none"> <li>Scripting files like PHP, database files, configuration files, rar files, executable files.</li> </ul>                                                                                                                                                                                                                                                                                                                                                                       |
| 3     | Port                  | <ul style="list-style-type: none"> <li>Port enables the communication between a device node and an execution environment node.</li> <li>It also enables the communication between a node and an external environment.</li> </ul>                                                                                                                                                                                                                                                                                   |

(Continued)

**Table 1.10 (Continued)** Components of UML Deployment Diagram

| S. No | Name of the Component | UML Notation and the Purpose of the Component                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|-------|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 4     | Association           | <ul style="list-style-type: none"> <li>It represents the two-way communication between any two nodes involved in a relationship.</li> </ul>  <p>The diagram illustrates an association between two nodes. On the left is a 'Device Node' containing an 'Execution Environment Node' which in turn contains an artifact labeled '&lt;&lt;artifact&gt;&gt; Artifact'. On the right is 'Device Node 1' containing an 'Execution Environment Node 1' which contains an artifact labeled '&lt;&lt;artifact&gt;&gt; Artifact 1'. A solid orange line connects the artifact in the first node to the artifact in the second node. A red label 'Association' with a dashed line points to this connection.</p> |
| 5     | Directed Association  | <ul style="list-style-type: none"> <li>It represents the one-way communication between any two nodes involved in a relationship.</li> </ul>  <p>The diagram illustrates a directed association between two nodes. The structure is identical to the previous diagram, with 'Device Node' and 'Device Node 1' each containing an 'Execution Environment Node' and an artifact. A solid orange line with an arrowhead at the end in the second node connects the artifact in the first node to the artifact in the second node. A red label 'Directed Association' with a dashed line points to this connection.</p>                                                                                     |
| 6     | Dependency            | <ul style="list-style-type: none"> <li>It represents how an existence of a node is dependent on another node involved in a relationship.</li> </ul>  <p>The diagram illustrates a dependency between two nodes. The structure is identical to the previous diagrams. A dashed orange line with an open arrowhead at the end in the second node connects the artifact in the first node to the artifact in the second node. A green label 'Dependency' with a dashed line points to this connection.</p>                                                                                                                                                                                              |

## *Chapter 2*

---

# **Design of UML Diagrams for Webmed – Healthcare Service System Services**

---

### **2.1 Problem Statement**

Healthcare service has huge demand these days as it really helps in managing a hospital or a medical office. The scope of healthcare service systems is increasing by each day and it is true for the entire world. Some of these solutions include improved awareness about healthcare services and health policies. The objective of this system is to provide medical assistance to people instantly with the help of technology. This system eradicates the cultural sensitivity that prevails in many hospitals and improvises the quality of medical assistance. The captivating features of this system are online doctors, medicines at doorstep and bulletin of awareness. The users can also navigate and choose among various insurance schemes that are displayed.

The primary objectives of Webmed healthcare system are to enable all citizens to receive healthcare services whenever needed, and to deliver health services that are cost-effective and meet pre-established standards of quality. The main functions of this system deal with finance, health A–Z, resources, drugs and supplements, news and experts, payment and feedback. Register function allows the patients or the caregivers to register on the website. Login function allows the patients to access the website. Financing focuses on the purchase of insurance. Health A–Z displays all the diseases along with their symptoms. Resources function consists of the sub-functions including symptoms checker, health calculator, find a doctor based on the geographical location of the patient, insurance guide and ambulance providence. Drugs and supplements include online medicine delivery, where people could shop for medicines online. News and experts function is to provide health awareness and threats that are prevailing. This function also gives information regarding counselling programs and blood donation camps. The payment function is to reimburse providers for services delivered. The feedback function collects user reviews for the website.

## 2.2 Major Functionalities and Its Description

The major functionalities of this system along with their description are as follows:

**Register** – this functionality acts as a membership functionality that allows the users, patients or caregivers to register into this website to access all the resources. **Login** – this functionality authenticates the user to provide access permissions. **Facilities** – this functionality allows us to access all the options available on the website. **Logout** – this functionality ends the access to the website. **Finance** – this functionality allows to purchase insurance, or to pay for healthcare services consumed. **Health A–Z** – this functionality provides details of all the diseases. **Resources** – this functionality has sub-functions, namely symptoms checker, health calculator, find a doctor, insurance guide and ambulance providence. **Drugs and supplements** – this functionality provides online delivery of medicines. This functionality takes the doctor’s statement for the issue of medicine and the patient’s willingness to buy it online. **News and experts** – this functionality provides health awareness and threats that are prevailing. **Payment** – this functionality deals with the payment for the services consumed. This functionality is split into two sub-functions namely, payment for doctors, for the services issued and payment for online med-delivery. **Feedback** – this functionality collects user reviews for this website.

**System Architecture:** System architecture is the conceptual model that defines the structure, behaviour and more views of a system. An architecture description is a formal description and representation of a system, organized in a way that supports reasoning about the structures and behaviours of the system. This architecture diagram shown in Figure 2.1 includes all the functionalities of the system in an ordered manner based on the sequence of occurrence of these functionalities. It represents the complete data flow starting from the register functionality through which the user could access the system to the logout functionality that ends the access

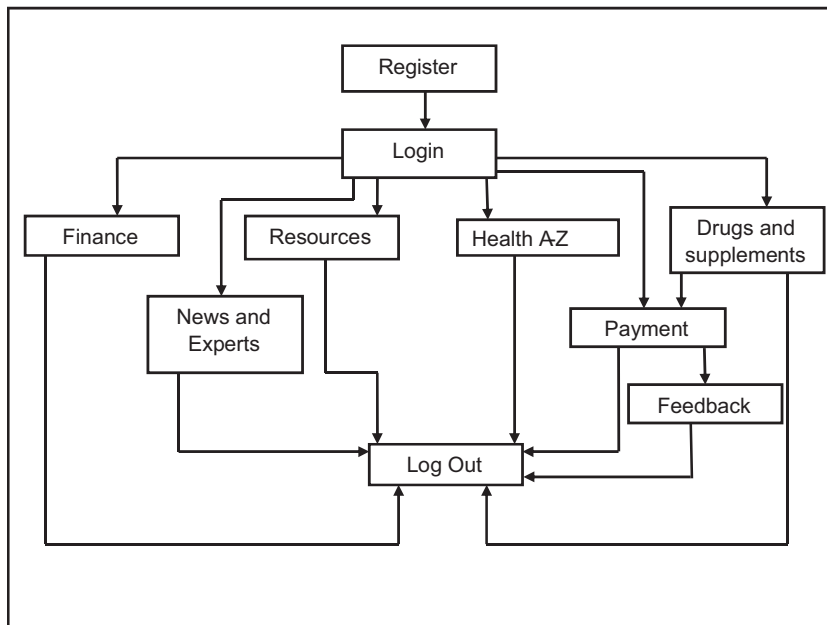


Figure 2.1 System architecture.

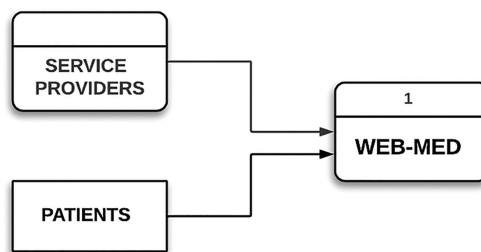
of the user to the system. Initially, the user must register onto the system. Once registered, the user would be able to log on to the system and could access all the services offered – finance, news and experts, resources that include finding a doctor, health A–Z that includes health calculator and fitness calculator, drug and supplements, payment and feedback. The payment is done through the insurance for the services consumed. After consuming the services, the user could log out of the system.

## 2.3 Data-Flow Diagrams

A data-flow diagram (DFD) is a way of representing a flow of a data of a process or a system. The DFD also provides information about the outputs and inputs of each entity and the process itself. A DFD has no control flow; there are no decision rules and no loops. A DFD can dive into progressively more detail by using levels and layers. DFD levels are numbered 0, 1 and 2.

DFD Level 0 is also called a Context Diagram. It is a basic overview of the whole system or process being analyzed or modelled. It is designed to be an at-a-glance view, showing the system as a single high-level process, with its relationship to external entities. This level 0 DFD diagram as shown in Figure 2.2 consists of the healthcare service provider system – Webmed and the actors who affect those systems – the users that is the patients and the service providers who are responsible for the maintenance of the system. This is the basic diagram that could be easily understood by a wide audience. DFD Level 1 as shown in Figure 2.3 provides a more detailed breakout of pieces of the Context Level Diagram. We will highlight the main functions carried out by the system, as you break down the high-level process of the Context Diagram into its sub-processes.

These systems too have the same external entities – patients who require the service and the service providers who offer those services via the system. The Webmed system is broken down into a number of sub-processes. These sub-processes represent the functionalities of the system – register, login, finance, health A–Z, resources, drugs and supplements, news and experts, payment, feedback and logout. Payment is done to find a doctor and the drugs and supplements services provided. The user could also give the feedback for the services available. Table 2.1 describes the comparison between the existing system and the proposed system.



**Figure 2.2** Level 0 DFD.



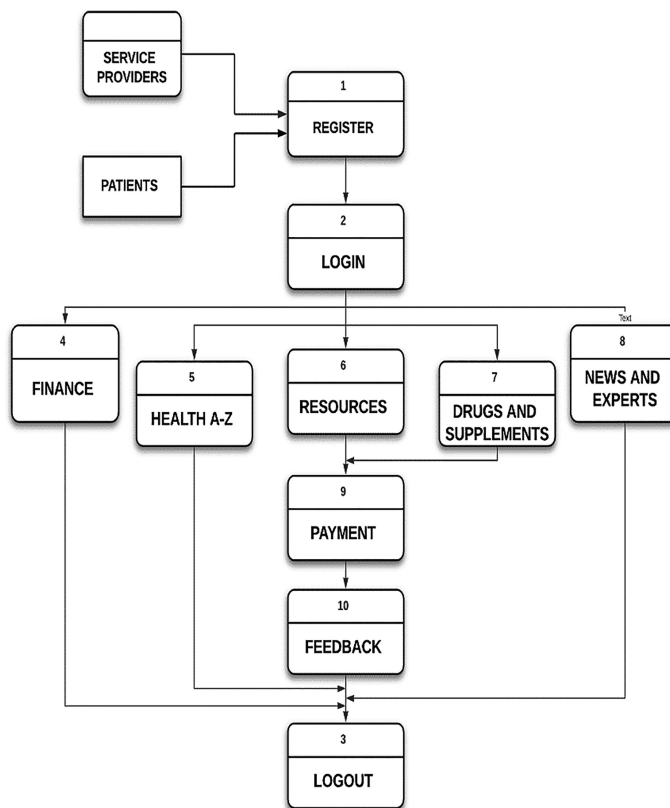


Figure 2.3 Level 1 DFD.

Table 2.1 Comparison of Proposed System and Existing System

| Existing System       | Additional Features of Proposed System with Reference to Existing System      |
|-----------------------|-------------------------------------------------------------------------------|
| Register              | The same as that of the existing system                                       |
| Login                 |                                                                               |
| Logout                |                                                                               |
| Finance               |                                                                               |
| Health A–Z            | It acts as a dictionary of diseases along with their symptoms.                |
| Resources             | The same as that of existing system.                                          |
| Drugs and supplements | This functionality provides online delivery of medicines.                     |
| News and experts      | This functionality provides health awareness and threats that are prevailing. |
| Payment               | The same as that of existing system.                                          |
| Feedback              | This functionality collects user reviews for this website.                    |

## 2.4 Proposed Design – UML Diagrams

### 2.4.1 Usecase Specification for the UML Usecase Diagram Generated Using Star UML Tool

#### Actor Specification

1. **Patient** – This actor plays the role of a patient who can register on the website and can make use of all the services provided by the website. This actor can also play a role of a caregiver.
2. **Service Provider** – This actor acts as the master of this website who provides certain services. This actor maintains a database of doctors that helps a patient to find a doctor and looks after the finance, insurance and payment requirements. He also gives descriptions regarding various diseases that can be referred by the users. He also provides a facility for the users to purchase drugs and other medicines online and spreads awareness through news regarding the prevailing threats (Figure 2.4).

#### Usecase Specification

##### 1. Drugs and Supplements

Description: This functionality provides online delivery of medicines.

Flow of Events

Basic Flow: It allows the users to search for and purchase drugs and other medical supplies like syringes, bandages and dressing, earloop face masks, etc.

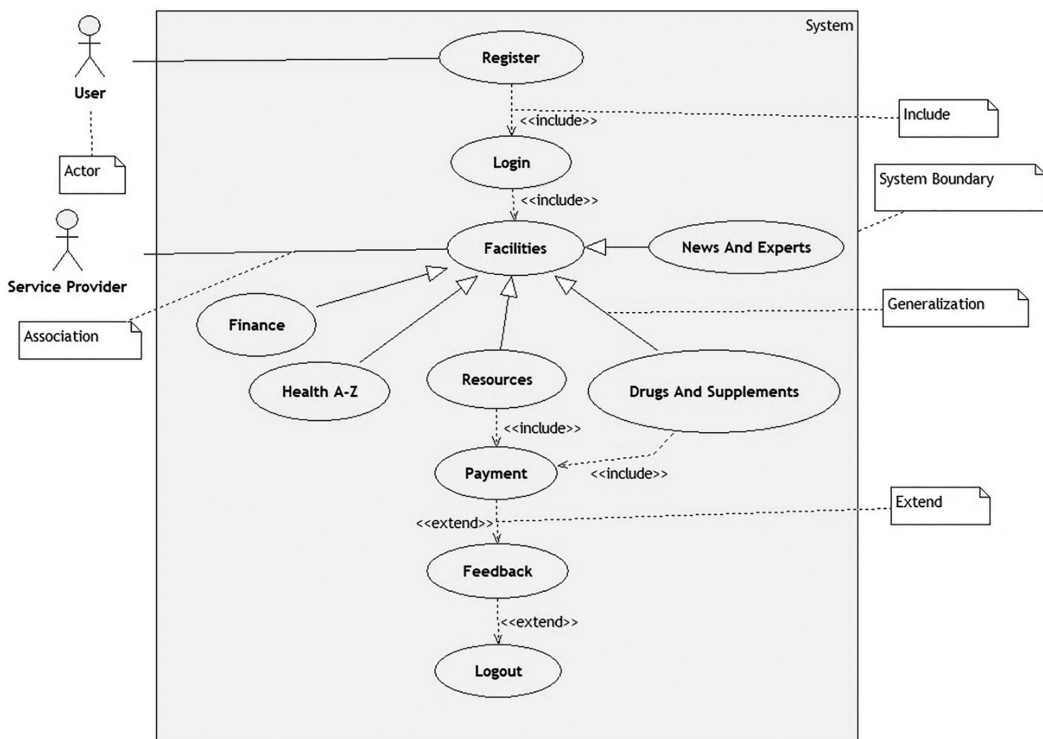


Figure 2.4 UML usecase diagram.

Alternative Flow: Alternate flow for this usecase is: Health A-Z, resources, finance, news, and experts.

Pre-Conditions: The user must have logged on to the website to make use of this functionality.

Post-Conditions: The drugs and medicines database get updated accordingly.

## 2. **Facilities**

Description: The service provider controls all the facilities on the website through this usecase and the user can utilize these facilities.

Flow of Events

Basic Flow: The user can obtain the services like: Health A–Z, resources, drugs and supplements, news and experts, and finance.

Alternative Flow: The user can provide his feedback for improvement of the website.

Pre-Conditions: The user must have logged on to the website in order to make use of this functionality.

Post-Conditions: By getting into this usecase, the user can make use of the available services.

## 3. **Feedback**

Description: This Feedback section helps the users to give their honest opinions about the quality of services that have been provided. It also helps them to give their suggestions for further improvements.

Flow of Events

Basic Flow: It verifies whether the feedback is given or not.

Alternative Flow: LOGOUT.

Pre-Conditions: The payment function should have been completed.

Post-Conditions: The feedback form is submitted.

## 4. **Finance**

Description: Negotiations regarding the financial status are done, eligibility for the insurance is checked and the appropriate schemes are issued.

Flow of Events

Basic Flow: It checks if the user has availed any insurance for his health if not it checks for insurance availability.

Alternative Flow: Instead of this usecase, the following usecases can be executed: Health A-Z, resources, drugs and supplements, news, and experts.

Pre-Conditions: The patients should have a valid bank account.

Post-Conditions: The patient would have insured his life.

## 5. **Health A–Z**

Description: This functionality acts as a dictionary of the diseases along with their symptoms and the drugs that can be consumed.

Flow of Events

Basic Flow: It provides catalogue of all the diseases with their symptoms.

Alternative Flow: Instead of this usecase, the following usecases can be executed: finance, resources, drugs and supplements, news, and experts.

Pre-Conditions: The user should have logged onto the website.

Post-Conditions: The user is now aware of the drugs that must be consumed for a specific disease.

## 6. **Login**

Description: This functionality allows the patients and caregivers to login into the website to get the access to all the resources.

#### Flow of Events

Basic Flow: It allows the user to access all the resources that are available in the website.

Alternative Flow: After this usecase is executed, then the control flows through any one of the following usecases: Finance, health A-Z, resources, drugs and supplements, news, and experts.

Pre-Conditions: The user should have registered into the website.

Post-Conditions: The user is given the complete access to all the resources available on the website.

### 7. Logout

Description: This functionality ends the access to the website. So the user must log in again to access all the resources.

#### Flow of Events

Basic Flow: It ends the access of the resources by the user.

Alternative Flow: It has no alternate flow.

Pre-Conditions: The user should have logged onto the website.

Post-Conditions: Once this usecase is executed, the user is denied from the resources provided by this website.

### 8. News and Experts

Description: A bulletin of news regarding the latest diseases that prevails in the environment, expert opinions to avoid those communicable and non-communicable diseases, awareness about the diseases will be displayed publicly.

#### Flow of Events

Basic Flow: The current affairs about the diseases are well received by the users.

Alternative Flow: Instead of this usecase, the following usecases can be executed: Health A-Z, resources, drugs and supplements, and finance.

Pre-Conditions: The user should have logged in.

Post-Conditions: The user is enlightened about the prevalent diseases.

### 9. Payment

Description: This functionality deals with the payment for the services consumed.

#### Flow of Events

Basic Flow: This functionality involves the payment for the services provided by the doctor.

Alternative Flow: After executing this usecase, the control moves to the usecase feedback.

Pre-Conditions: The user should have consumed the services provided by the doctor or should have purchased medicines online.

Post-Conditions: After executing this usecase, the user has made the payment for the medicines purchased or the service providers have made the payment to the doctors.

### 10. Register

Description: This acts as a membership functionality that allows the patients or caregivers to register into this website to access all the resources

#### Flow of Events

Basic Flow: It allows the user to register on the website.

Alternative Flow: After the register usecase is executed, the usecase that can be executed is: LOGIN functionality.

Pre-Conditions: This is no precondition as register is the first usecase to be executed.

Post-Conditions: After the register usecase is executed, the user will be given a Webmed membership.